TECHNICAL ASSESSMENT

WordPress Petstore Directory Plugin

Prepared for prospective WordPress development vendors — DBSync Inc.

Time-box	No more than 2–6 hours total
Submission Deadline	24th July | Friday EOD by 5pm PST
Submit To	https://forms.gle/MbHtGE2awBzkb3979
RFP Tech Assessment	https://docs.google.com/document/d/1p7MRDg0q0RuCopXAO5itcQD3zKKNhp6jmquvYsSUeDw/edit?usp=sharing

Objective

Build a custom WordPress plugin using Claude that pulls pet listings from the public Swagger Petstore API and displays them in a formatted table via a custom Elementor widget.

This exercise is not expected to be production-polished. We are evaluating your approach, judgment, and process at least as much as the finished result.

API Reference

- Swagger 2.0: https://petstore.swagger.io/v2/swagger.json
- OpenAPI 3.0: https://petstore3.swagger.io/api/v3 (either version is acceptable)
- Use the find-by-status endpoint (e.g., GET /pet/findByStatus)

Functional Requirements

1. Custom plugin — proper plugin header, activation/deactivation hooks as appropriate. Not a snippet dropped into functions.php.
2. Data retrieval — fetch pets from the API (name, category, status, and photo URL where available).
3. Elementor integration — build a custom Elementor widget (extending \Elementor\Widget_Base) that:
    - Appears in the Elementor editor panel under a custom category
    - Renders the pet list as a table
    - Exposes at least 2–3 style controls in the Elementor panel (e.g., border color, table header background, rows-per-page) — not just a shortcode dropped into a text box. We want to see real widget extension, not a workaround.

4. Admin settings page — API base URL and default status filter configurable without touching code.
5. Caching — use WordPress transients (or similar) to avoid calling the external API on every page render.
6. Error handling — a graceful front-end message if the API is unreachable or returns an error.
7. Security — input sanitization, output escaping, and nonce verification on the settings form.

Note: build against free Elementor. No Elementor Pro-only features are required or expected.

A Note on Requirements

Some of the requirements above are in tension with each other. This is intentional and not called out further — part of what we're evaluating is whether you notice it, how you think about the tradeoff, and what you decide to do about it. There is no single correct resolution; we want to see your reasoning, documented in your decision log below.

Process Requirements

- Work in the Git repository we provide, with a visible, logically incremental commit history — not a single "final" commit.
- At least one feature branch, merged via a pull request with a short description.
- Commit messages should briefly note why, not just what (e.g., "cache API responses — avoid rate limiting on repeated page loads").

Deliverables

1. **Record the exercise as you develop and upload in the submission. Make sure recording is full screen.**
2. Git repository (or export with full history intact) — code and commit history
3. Installable plugin .zip
4. Short README — setup, widget usage, configuration
5. Decision log (half a page maximum) answering:
    a. What tradeoff(s) did you notice in the requirements, and how did you resolve them?
    b. Why did you choose your caching approach over alternatives?
    c. What's the first thing that would break under real-world load, and why?
    d. What would you do differently with more time?
6. AI collaboration transcript — if you used Claude, Copilot, or similar, export and include your chat/prompt history. We're looking at how you worked with the tool (did you verify, correct, or push back on its output), not whether you used one — using AI well is expected.
7. **Submit at https://forms.gle/MbHtGE2awBzkb3979**